Manual técnico

Robot de laberinto · Microcontroladores 2

Manuel Harari López
Víctor Miguel Vázquez
Carlos Emiliano Escamilla Garibay

Cómo leerlo. Cada sección empieza con un recuadro **En simple**, que se entiende sin ser ingeniero. Lo que sigue es el detalle técnico. Al final hay un glosario.

1. Qué hace el robot

En simple: el robot entra a un laberinto que no conoce y avanza celda por celda, mirando con tres sensores láser dónde hay pared. Busca el cuadro **amarillo** del piso, que es la meta, y se detiene al verlo. En el camino cuenta los cuadros de colores y dibuja un mapa. En el siguiente intento usa ese mapa para ir directo a la meta por el camino más corto que conoce.

- **Laberinto:** celdas de 180 mm. La meta es el cuadro amarillo; no se supone un tamaño de laberinto fijo.
- **Robot:** diferencial con dos ruedas motrices y un patín, huella ≤ 120 mm.
- **Estrategia en dos fases:**
  1. **Exploración** guiada por los sensores: regla de la mano izquierda: izquierda si se puede, si no de frente, si no derecha, si no media vuelta. El mapa se va llenando con votos.
  2. **Ruta corta** en la siguiente corrida: si el mapa ya tiene la meta, va por el camino más corto conocido (BFS). Si algo no cuadra, sigue explorando desde donde quedó.

2. Dos cerebros

En simple: el Arduino UNO Q trae dos computadoras en una placa.

- Una **pequeña y muy puntual**, que mueve los motores y lee los sensores cien veces por segundo.
- Una **grande con Linux**, que decide el camino, guarda el mapa y mira con la cámara.

Se pasan mensajes entre ellas. El STM32 ejecuta cada movimiento por su cuenta y frena solo (a 35 mm de una pared, o al ver amarillo), así que el robot nunca queda fuera de control si Linux tarda.

Dominio	Procesador	Sistema	Tareas
Tiempo real	STM32U585 (Cortex-M33)	Zephyr (sketch Arduino)	Motores, 3 ToF, clasificación de color, MPU-6050 opcional, pantalla y botón
Decisión	Qualcomm QRB2210 (Cortex-A53)	Debian Linux	Máquina de estados, mapa, ruta corta, persistencia, cámara (OpenCV), página web

Enlace: el **Bridge** (RPC MessagePack, enrutado por `arduino-router`). Linux invoca

procedimientos que expone el STM32:

Procedimiento	Qué hace
`mover(mm, lento)`	Avanza hasta *mm* (una celda = 180 mm). Si `lento` es 1 usa `VEL_LENTO`. No bloquea; devuelve 0 si ya hay un movimiento en curso
`girar(k)`	Gira en sitio *k* × 90°; el signo indica el sentido (2 = media vuelta)
`ajustar(g)`	Corrige el rumbo *g* grados (derecha +) con lo que mide la cámara. Con MPU y sentido conocido no gira: mueve el rumbo deseado y la recta lo sostiene (devuelve 1). Si no, gira en su lugar (devuelve 2: hay que esperar)
`alinear(lim)`	Con MPU: gira en su lugar hasta el rumbo deseado si se desvió 3° o más (`ALIN_MIN`) y no más de *lim* (45 antes de avanzar; 180 para terminar un giro fallido). Devuelve 2 si hay que esperar. Sin MPU devuelve 0
`ocupado()`	1 mientras hay un movimiento activo
`odo(i)`	Lecturas: 0 = mm recorridos, 1 = mm que faltan, 2 = máscara de paredes y movimiento, 3 = saltos (1 izquierda, 2 frente, 4 derecha), 5 / 6 / 7 = distancia frente / izquierda / derecha (mm), 8 = desvío de rumbo en grados, 12 = el último avance chocó, 13 = veces que el último giro se atoró, 14 = lo levantaron (se borra al leerlo), 15 = golpe máximo del último avance en mg (12 a 15 solo con MPU)
`color()`	Último color confirmado (3 lecturas seguidas), una sola vez
`alto()`	Detiene el movimiento (Python lo pide al confirmar la meta)
`arranque()`	Evento del botón: 1 = corto, 2 = largo
`set_cuenta` , `set_estado` , `set_fase`	Actualizan la pantalla (conteo, estado, fase de la máquina)
`raw(i)`	Lectura cruda del sensor de color (*i* = R, G, B, C) para el panel web
`set_ref(i, r, g, b, c)`	Carga el color de referencia *i* (r, g, b en milésimas)

Procedimiento	Qué hace
`set_tof(i, a, b)`	Carga la calibración del ToF *i*: real = *a* × crudo + *b* (*a* en milésimas, *b* en décimas de mm). `odo(9..11)` da la lectura sin calibrar

2.1 Página web: calibración y diagnóstico

Linux sirve una página en el puerto 7000 (brick `web_ui`), que se abre desde el celular por WiFi. Es la herramienta central de puesta a punto: toda la calibración se hace ahí, sin reprogramar ni copiar valores a mano, y queda guardada en `/app/datos/` para cargarse sola en cada arranque.

Función	Cómo funciona	Archivo
Colores	Asistente de 5 pasos; cada color promedia 5 lecturas del TCS34725 y se manda al STM32 (`set_ref`)	`colores.json`
Distancia	Por sensor, puntos (mm reales con vernier, lectura cruda promedio de 10); en pista se usa 1 punto a 100 mm por sensor: solo desfase `real = crudo + b` (con varios puntos el código ajusta `real = a × crudo + b` por mínimos cuadrados) que se manda al STM32 (`set_tof`)	`tof.json`
Simulador	Dibuja lo que mide cada ToF y lo que haría el robot (`que_haria()`) sin moverlo	
Cámara	Imagen 160 × 120 en gris con la zona de paso en rojo (blanco) o verde (negro)	
Corrida	Fase, celda, mapa y meta; durante la corrida usa la última lectura del robot y no llama al Bridge	

Se aplica en el STM32, antes de cualquier decisión, para que el control de 10 ms, los saltos y el alto a 35 mm usen milímetros reales.

Separar los dominios aísla la latencia no determinista de Linux (planificador, E/S, visión) del lazo de control.

Respaldo: si el UNO Q falla, el mismo chasis acepta un UNO R3 (ATmega328P) con firmware *bare-metal* propio (carpeta `atmega328p/`) que hace todo en un solo chip, sin Arduino core. Este manual describe la versión del UNO Q.
Un solo sketch, con MPU-6050: `uno_q/sketch/`. La versión sin MPU (`sketch_sin_mpu/`) se retiró del repositorio para no confundir; queda en el historial de git (ver la sección "Control de versiones").

3. Sensores de distancia

En simple: tres sensores láser (izquierda, frente y derecha) miden en milímetros qué tan lejos está la pared, unas 50 veces por segundo. Una lectura rara nunca decide sola: el robot toma varias y se queda con la menor.

- **Sensores:** 3 × VL53L0X, tiempo de vuelo a 940 nm.
- **Direcciones I²C:** los tres traen la misma (0x29), que además comparte el sensor de color. Al arrancar se mantienen apagados con XSHUT (A0–A2). Se encienden uno por uno y cada uno se reasigna a 0x30–0x32 antes de `init()`.
- **Lectura:** modo continuo con *timing budget* de 20 ms (≈ 50 Hz por sensor), no bloqueante.
- **Umbrales:** lateral con menos de 120 mm (`PARED`) = pared. Frontal con menos de 55 mm reales (`PARED_F`) = pared en su celda; con 130 mm o más (`FRENTE_LIBRE`) el frente cuenta como salida.
- **Lecturas mínimas:** Python toma 3 lecturas de cada ToF y usa la menor (5 tras un cambio brusco). Una lectura mala no deja pasar al robot.

4. Sensor de color

En simple: debajo del robot hay un sensor con su propia lucecita, tapado con una capucha para que no le afecte la luz del lugar. Mide cuánto rojo, verde y azul refleja el piso y lo compara con los colores que le enseñamos en la calibración. Para no contar dos veces el mismo cuadro, exige ver el mismo color tres veces seguidas y no vuelve a contar hasta regresar al piso normal. El amarillo es la meta: el robot se detiene en cuanto lo confirma.

Adquisición

- TCS34725 con integración de 24 ms, ganancia 4× y LED propio, dentro de una capucha impresa.
- La capucha fija la distancia al piso y bloquea la luz ambiente.

Clasificación por vecino más cercano. Primero se normaliza la cromaticidad, que da invariancia de primer orden a la intensidad de luz:

> $\hat{r} = R / (R + G + B)$, igual para $\hat{g}$ y $\hat{b}$

La distancia a cada color de referencia *i* suma un término logarítmico de brillo. Ese término separa colores de tono parecido y distinta reflectancia, como blanco y amarillo:

> $d_i = (\hat{r} - r_i)^2 + (\hat{g} - g_i)^2 + (\hat{b} - b_i)^2 + 0.01 \cdot \ln^2((C + 1) / (C_i + 1))$

El resultado es “piso” si la menor distancia es mayor que 0.02 o si el brillo es menor que 1.3 veces el del piso negro (`BRILLO_MIN`, quita falsos positivos).

Calibración: desde un panel web que sirve la propia placa (puerto 7000). Cada referencia es el promedio de 5 lecturas tomadas sobre la pista. Se guarda en `/app/datos/colores.json`; si no existe, se usan los valores de fábrica. Linux la carga al STM32 con `set_ref` en cada arranque, sin recompilar el sketch.

Antirrebote: se exigen 3 muestras iguales seguidas (3 × 30 ms). El evento solo se dispara en la transición piso → color.

Meta: al confirmar amarillo, el **STM32 detiene el movimiento por sí mismo**, sin esperar a Python. Python entonces guarda la celda como meta.

¿En qué celda estaba? Python calcula $k = \lfloor (mm + 36 + 90) / 180 \rfloor$, con *mm* los milímetros recorridos en el avance y 36 mm la distancia del eje al sensor (`Y_COLOR`). Cada celda cuenta su color una sola vez.

5. Cámara (opcional)

> **En simple:** la cámara va arriba de un poste y mira el piso por donde va a pasar. Es **monocromática** (ve en gris), y el piso negro con paredes blancas es justo lo que mejor distingue. Antes de cada avance toma **una foto** y hace tres cosas: revisa si hay blanco (pared) en el camino para ir más despacio, mide si el robot va chueco viendo hacia dónde se juntan las dos paredes del pasillo, y lo endereza. La cámara nunca manda sobre los láser: solo **asiste**. Sin cámara, el robot funciona igual.

Sensor. innomaker U20CAM-9281M: OV9281 de 1 MP **monocromático** con **obturador global** (todos los píxeles se exponen al mismo tiempo), hasta 720p a 120 fps, lente M12 gran angular, UVC (Linux la ve sin driver). Se captura a 320 × 240 con OpenCV y V4L2. El obturador global evita la deformación por movimiento (*rolling shutter*) de las cámaras comunes, que tuerce las

líneas rectas cuando el robot gira. Al no tener filtro de color (matriz de Bayer), cada píxel recibe toda la luz: más sensibilidad y menos ruido en la luminancia, que es lo único que se usa. Por la misma razón **no puede ver el amarillo**; la meta la confirma el TCS34725.

Versiones. Un solo Python (`python/main.py`) con una bandera, `EXPERIMENTAL`. En `False` (como está en git) usa las técnicas 1 a 3; en `True` agrega la 4 y la 5:

`EXPERIMENTAL` en `python/main.py`	Cámara
`False` (estable, como está en git)	1 a 3: exposición fija, umbral automático (Otsu), corrección de rumbo
`True` (experimental)	1 a 5: además, distancia por cámara y aberturas adelante

5.1 Técnicas 1 a 3 (versión estable)

1. **Exposición fija.** Con exposición automática, la cámara cambia su ganancia según lo que tenga enfrente (una pared blanca cerca la oscurece, la ropa de la gente la aclara), y un mismo piso cambia de gris entre fotos. Desde la web se fija la exposición (`CAP_PROP_AUTO_EXPOSURE` en manual, `CAP_PROP_EXPOSURE` en unidades UVC de 100 µs) y se guarda en `/app/datos/camara.json` .
2. **Umbral automático (Otsu) con contraste y sensibilidad.** Antes de Otsu, el gris se **estira**: el 2 % más oscuro de la mitad de abajo pasa a 0 y el 2 % más claro a 255 (si la imagen es casi uniforme no se estira, para no inventar contraste). Con poca luz o exposición baja, piso y pared quedan bien separados. La **sensibilidad** (web, de −80 a 80) baja el umbral de Otsu esos niveles de gris: más alta, ve blanco antes. El histograma de gris de la escena es **bimodal**: piso negro y pared blanca. El método de Otsu elige el umbral que maximiza la varianza entre las dos clases, en cada foto, sin calibrar un número. Se calcula sobre la mitad de abajo de la imagen (el piso cercano) después de un filtro de mediana de 5 × 5, que quita reflejos de un píxel. Para que una imagen toda negra no “invente” blanco, el umbral se acota a [`UMBRAL_MIN` , `UMBRAL_MAX`] = [100, 220]. Con él se mide la fracción de blanco en `ZONA_PASO` : si pasa de `BLANCO_MAX` (15 %), Python pide `mover(mm, 1)` (lento) y lee el frontal como doble check; el sketch para a 35 mm si la pared es real.
3. **Rumbo por punto de fuga.** En cada fila de `ZONA_BORDES` (45 % a 95 % de la altura) se busca, desde el centro hacia cada lado, el primer píxel blanco: la base de la pared izquierda y la derecha. Cada pared se ajusta a una recta $x = a \cdot y + c$ por mínimos cuadrados con rechazo de atípicos (empieza con la mitad más cercana, la más confiable, y descarta los puntos a más de 5 px, como el fondo de un pasillo lateral). Las dos rectas, paralelas en el piso, se cortan en la imagen en el **punto de fuga** del pasillo. Con una cámara de proyección central, la columna del punto de fuga depende **solo del ángulo** entre el robot y el pasillo, no de la posición lateral del robot ni de la inclinación de la cámara: $\theta = atan(k \cdot (x_f - x_0))$, donde x_0 es la columna con el robot derecho y k los radianes por píxel. Las dos se miden en la web (robot derecho, y girado 15° a mano), así que no hace falta conocer la inclinación de la cámara ni su lente.

- Si |θ| ≥ `ANG_MIN` (4°), Python llama `ajustar(θ)` antes de avanzar. Si |θ| > `ANG_MAX` (20°), la lectura se descarta como mala.
- **Con MPU-6050:** no gira. El giroscopio deriva con el tiempo, y la cámara ve el error real respecto al pasillo; `ajustar` mueve el rumbo deseado (`ref`) y el control de recta (`KG`) lo corrige durante el avance, sin perder tiempo. La cámara corrige la deriva de la integral y el giroscopio da la respuesta rápida, como un filtro complementario.
- **Si el MPU no respondió al arrancar** (sin `G` en la OLED): gira en su lugar por tiempo (`MS_90` × θ / 90, mínimo 30 ms). Es un respaldo aproximado.
- En pruebas con imágenes sintéticas de un pasillo (cámara a 150 mm de altura, 35° hacia abajo, ruido de ±6 niveles de gris), el ángulo medido coincidió con el real dentro de ±0.1° entre −12° y 12°, también con el robot descentrado 25 mm y con una pared abierta.

> Un intento anterior de alinear con giros de 5° tomando muestras de los ToF volvió al robot muy lento y se quitó. Aquí es una sola foto por celda y, con MPU, no se gira: solo se cambia la referencia.

5.2 Técnicas 4 y 5 (versión experimental)

4. **Distancia por cámara (“ToF virtuales”).** En el piso, una fila de la imagen corresponde a una distancia hacia adelante (geometría de un plano visto por una cámara fija). En lugar de modelar altura, inclinación y distorsión de la lente gran angular, se **calibra con el ToF frontal como regla**: con el robot frente a una pared a varias distancias, la web guarda la fila de la base de la pared en la columna central y la lectura del frontal (`/app/datos/camara.json`). Después, en 5 columnas (`COLS` = 10, 30, 50, 70 y 90 % del ancho) se busca la base de la pared desde abajo y se interpola su distancia. Las columnas laterales ven en diagonal, justo donde los 3 ToF no ven (lo que se buscaba con ultrasónicos a 45°). Fuera del rango calibrado no da valor. En pruebas sintéticas la columna central midió 209, 306, 400 y 517 mm con paredes a 200, 300, 400 y 500 mm.
5. **Aberturas en la celda siguiente.** Con la tabla anterior se sabe qué filas corresponden a las paredes laterales de la celda siguiente (`SIG` = 60 a 240 mm del frontal). Si en ese tramo menos del 30 % de las filas tiene la base de la pared sobre la recta de esa pared, ese lado está abierto (sin blanco, o el blanco es la pared del fondo del pasillo lateral). Entonces el robot avanza **lento**, para que la detección de saltos del sketch lo detenga centrado, y anota un **voto débil** (−1) de libre en esa celda: si los ToF dicen lo contrario al llegar, ganan ellos. En pruebas sintéticas detectó la abertura izquierda, la derecha (imagen espejo) y la izquierda con el robot girado 6°, sin falsos positivos en el pasillo cerrado.

Son experimentales porque dependen de que la calibración por filas sea estable y de cómo se ve la base de las paredes reales (sombras, uniones, madera). Se prueban primero mirando la web con el robot quieto.

5.3 Montaje y web

- **Montaje:** módulo (medida por confirmar; el CAD usa 32 × 32 mm) sobre un pedestal impreso, inclinado hacia abajo para ver el piso adelante del eje. La inclinación quedó la de la primera versión del soporte (la última del CAD no se imprimió); como todo se calibra con el robot real, no afecta. Se conecta por un hub USB-C con entrada de alimentación: con la Q alimentada por VIN, su USB-C no da 5 V al hub, así que el hub se alimenta por su puerto PD.
- **Web:** la imagen a 160 × 120 (JPEG calidad 40) muestra el camino en rojo (blanco) o verde (negro), las bases de las paredes y el punto de fuga en azul, la línea de "derecho" en blanco y, en la experimental, la base de la pared por columna en amarillo. Abajo: porcentaje de blanco, umbral de Otsu, rumbo, distancias y aberturas. "Calibrar cámara" tiene los 4 pasos. Un candado comparte la cámara entre el robot y el panel. Al arrancar la busca una vez; un hilo revisa cada 2 s si sigue conectada (OLED `SIN CAM`).

6. Cómo se mueve

> **En simple:** el robot no tiene sensores en las ruedas, así que mide cuánto avanza con el láser de enfrente: anota a qué distancia está la pared al arrancar y para cuando esa distancia bajó 18 cm (una celda). Si no hay pared a la vista, avanza por tiempo. Para los giros usa un giroscopio si lo tiene, y si no, tiempo. Frena siempre a 13 cm de una pared.

Actuación

- Dos micromotorreductores DC (≈ 12 × 11 mm de sección), ruedas de 45 mm, puente H L298N.
- PWM en ENA/ENB y sentido en IN1–IN4. El sentido de cada motor se invierte por software (`INV_IZQ`, `INV_DER`).
- Freno activo: ambas entradas en alto con EN = 255 durante 60 ms.

Avance (lazo a 100 Hz, $T = 10$ ms):

1. **Por distancia.** Al iniciar toma el frontal; si lee menos de 1 m (`RANGO`) avanza hasta que haya bajado 180 mm (`CELDA`). Si no ve nada a ese alcance avanza por tiempo (`MS_CELDA`) y, si a medio camino aparece una pared, sigue por distancia.
2. **Alto y retroceso.** Se detiene al leer 35 mm reales al frente (`D_FRENTE`, frontal calibrado). Si queda por debajo de 30 mm (`D_FRENTE − D_CERCA`) se hace para atrás hasta 35 mm. Si hay pared y la distancia ya no baja en 300 ms, también para. Desde 135 mm frena de forma proporcional (`FRENA_K`).
3. **Perfil de velocidad.** Rampa de +6 PWM por ciclo hasta `VEL_EXPLORA` (o `VEL_LENTO`), y desaceleración lineal en el último cuarto de celda, con piso en `VEL_MIN`.
4. **Cambios bruscos.** Si una lectura de un ToF cambia `SALTO` = 120 mm o más y la siguiente lo confirma, se reconoce un cambio brusco (se abrió un lateral o saltó el frontal).

    - Si se abre un lateral, el robot avanza `TRAS_SALTO` = 124 mm más y para en el centro de esa celda, para escanearla (fase `ESCANEA`).
    - Si salta el frontal, recalcula lo que falta con la nueva lectura.
    - Python lo lee con `odo(3)`.
5. **Centrado solo bajo 90 mm.** Con un lateral a menos de `D_LADO_MIN` = 90 mm corrige: $u = K_c \cdot e + K_d \cdot \dot{e}$, saturado en ±40 PWM y aplicado en diferencial ($PWM_izq = v - u$, $PWM_der = v + u$). Con dos paredes cercanas $e = d_izq - d_der$; con una, contra los 90 mm. Con ambos lados lejos va derecho.

Giros

- **Con MPU-6050:** el sketch lo busca en la dirección 0x68 al arrancar (OLED muestra `G` si lo encontró) y mide el sesgo del giroscopio con el robot quieto, por lo que se enciende sin moverlo.
    - **Rumbo absoluto:** cada giro va al ángulo deseado acumulado, así corrige lo que se pasó o faltó en el giro anterior. Se corta `ADELANTO` = 6° antes por la inercia.
    - **Aprende el signo:** el sentido de montaje del sensor se determina en el primer giro. Antes de ese giro no sostiene rumbo.
    - **Rumbo en recta:** durante el avance sostiene el rumbo con ganancia `KG`, además del centrado con paredes.
- **Respaldo si el MPU no responde al arrancar:** gira por tiempo (`MS_90`) a PWM constante `VEL_GIRO`.

6.1 MPU-6050: giroscopio y acelerómetro

> **En simple:** el MPU-6050 es un chip que siente **giros** (giroscopio) y **empujones** (acelerómetro). Con el giroscopio el robot sabe exactamente cuántos grados ha girado y hacia dónde debe apuntar; con el acelerómetro se da cuenta si chocó o si alguien lo levantó. Con él, el robot gira a 90° de verdad, avanza derecho, se endereza antes de cada avance y se recupera solo de los choques.

Conexión. Módulo GY-521 en el bus I²C (dirección 0x68, AD0 a GND), VCC a 3.3 V. Va atornillado **plano** sobre el piso del chasis: el eje Z queda vertical y es el que mide los giros. La orientación en el plano no importa: el signo lo aprende en el primer giro.

Configuración (`mpuInit`): sale del modo de reposo (registro 0x6B = 0), filtro digital de ~44 Hz (0x1A = 3), giroscopio a ±500 °/s (0x1B = 0x08, 65.5 LSB por °/s) y acelerómetro a ±2 g (0x1C = 0, 16384 LSB/g). Con el robot quieto promedia 200 lecturas del giroscopio Z para restar su **sesgo** (*bias*); por eso hay que encenderlo sin moverlo.

Rumbo. Cada 2 ms integra la velocidad angular: $\psi \leftarrow \psi + (g_z - b) / 65.5 \cdot \Delta t$. El sketch guarda un **rumbo deseado absoluto** (`ref`) que cada giro aumenta en ±90°. Así el error de un giro no se acumula: el siguiente gira lo que haga falta para llegar a `ref`.

Función	Cómo	Parámetro
Giro por ángulo	Gira hasta `ref`; baja la velocidad en los últimos 25° y corta `ADELANTO` antes por la inercia	`ADELANTO` = 6°
Rumbo en recta	Suma $K_G \cdot (\psi - ref)$ (saturado a ±20 PWM) a la corrección de centrado	`KG` = 2.0 PWM/°
Alinear antes de avanzar	`alinear(45)`: si el rumbo se desvió 3° a 45°, gira en su lugar hasta `ref` antes de arrancar	`ALIN_MIN` = 3°
Verificar cada giro	Python lee el desvío (`odo(8)`); si pasa de 10° termina el giro con `alinear(180)` (2 intentos). Si aun así quedó 45° o más fuera, acepta el rumbo real: corrige el mapa en cuartos de vuelta y mueve `ref`	—
Giro atorado	Si girando pasan 400 ms sin 2° de giro, una esquina está trabada: retrocede 200 ms y sigue (hasta 2 veces)	`ATORADO_MS` = 400 ms
Choque	Cada 10 ms mide la aceleración horizontal menos un promedio lento (quita la inclinación del montaje). Más de 0.6 g en un avance, pasados los primeros 150 ms, es un choque: retrocede 250 ms y termina. Python lo registra y se endereza antes de seguir	`CHOQUE_G` = 0.6 g, `RETRO_MS` = 250 ms
Levantado	Si la aceleración vertical baja de 0.8 g durante 150 ms (inclinado o en el aire), apaga los motores y Python termina la corrida	`LEVANTA_G` = 0.8 g
Fusión con la cámara	La cámara mide el ángulo real respecto al pasillo y `ajustar` mueve `ref`: corrige la deriva lenta del giroscopio, que da la respuesta rápida (filtro complementario)	`ANG_MIN` = 4°

Lecturas para depurar (`odo`): 8 = desvío en grados, 12 = el último avance chocó, 13 = veces que el último giro se atoró, 14 = lo levantaron, 15 = golpe máximo del último avance en mg (sirve para calibrar `CHOQUE_G` con el log).

Calibración en pista. `ADELANTO`: si se pasa del giro, subirlo de 2 en 2; si le falta, bajarlo. `KG`: si zigzaguea en recta, 1.5; si tarda en enderezarse, 3. `CHOQUE_G`: si detecta choques sin chocar, subirlo; si no los detecta, bajarlo, con el golpe que muestra el log. `LEVANTA_G`: si se detiene solo, bajarlo a 0.7.

> **Limitación reconocida:** sin encoders, la distancia depende del frontal. En tramos largos sin pared a la vista el resultado depende del voltaje de la batería y de la fricción.
>
> - **Cómo lo mitigamos:** el frontal mide el avance por distancia, el MPU-6050 corrige el rumbo, y las ruedas patinan poco por la rampa de arranque.
> - **Mejora natural:** encoders en cuadratura para medir también los tramos sin pared.

7. Cómo decide y qué mapa guarda

> **En simple:** el robot decide siempre quieto, en cuatro pasos que se repiten: **mira** (con los láser), **decide** (a dónde ir), **gira** y **avanza** una celda. Sigue la regla de la mano izquierda: si puede, gira a la izquierda; si no, sigue de frente; si no, a la derecha; y si está encerrado, da media vuelta. Prefiere siempre los lugares donde no ha estado; si ya no hay ninguno cerca, regresa por donde conoce al lugar más cercano que le falte explorar. Así entra a la meta del centro aunque esté rodeada de paredes.

Máquina de estados (Python; la fase se ve en la OLED y en la web):

`SENSA` → `DECIDE` → `GIRA` → `AVANZA` → `SENSA`. Tras un cambio brusco, `ESCANEA` sustituye a `SENSA`.

Fase	Qué hace
`SENSA`	Toma 3 lecturas de cada ToF (la menor) y vota las paredes de la celda
`ESCANEA`	Igual, con 5 lecturas, tras un salto
`DECIDE`	Elige la salida (ver abajo)
`GIRA`	Pide `girar(k)` y espera
`AVANZA`	Pide `mover(180, lento)` y espera; lee mm recorridos y saltos

Salidas libres (todas con lecturas de ToF), en orden de mano izquierda:

1. **Izquierda** si el lateral lee ≥ 120 mm.

2. **Frente** si el frontal (calibrado) lee ≥ 130 mm (`FRENTE_LIBRE`): a la mitad entre la pared de su celda (~35-45 mm reales) y la de la siguiente (~215-225).
3. **Derecha**, con el mismo criterio que la izquierda.
4. **Media vuelta** si no hay ninguna.

Exploración completa (tipo Trémaux / búsqueda en profundidad). De las salidas libres elige la primera, en ese orden, que lleve a una celda **sin ver** (`vistas`: celdas donde ya leyó sus 3 ToF; se guardan con el mapa). Si todas llevan a celdas vistas, busca con BFS, por pasos conocidos, la celda vista más cercana que tenga un lado libre hacia una celda sin ver (`frontera()`), y da el primer paso hacia ella. Así recorre todas las celdas alcanzables.

Por qué. La meta está al centro y solo tiene 2 entradas: las paredes que la rodean forman una **isla**, separada del muro exterior. Un seguidor de pared (mano izquierda o derecha) solo recorre el muro al que está pegado, así que rodea la isla para siempre sin entrar. En simulación (laberintos de 9 × 9 con la meta en una isla central de 3 × 3 con 2 entradas, 30 laberintos), la mano izquierda pura llegó a la meta 0 de 30 veces y la exploración completa 30 de 30.

Historia: antes se probó “frente primero, luego izquierda”. En pista se atoraba en una zona: preferir el frente rompe la garantía del seguidor de pared, que solo recorre todo el muro conexo si **siempre** prefiere el lado de su mano. Con la mano izquierda pura esa garantía vuelve.

Tras un salto (cambio brusco en un ToF) escanea con 5 lecturas en lugar de 3 y decide con la misma regla.

Mapa por celdas. Cada celda guarda 4 votos (N, E, S, O) en un diccionario de coordenadas.

- Cada lectura vota pared (+1) o libre (−1), con tope ±5 (`VOTO_MAX`). Así una lectura mala no borra lo que vieron las demás.
- Por donde el robot pasó queda libre con certeza.
- La celda actual sale de los mm que de verdad avanzó (no de lo que pidió).
- Cada color leído se anota en su celda; el amarillo marca la meta.

Persistencia. Al terminar la corrida se guarda `/app/datos/mapa.json` (paredes, colores y meta), con escritura atómica. Al leerlo tolera un archivo roto o vacío. Un toque largo del botón (más de 2 s) borra mapa y conteo.

Ruta corta. Con la meta en el mapa, la siguiente corrida calcula por BFS el camino más corto sobre pasos conocidos como libres (OLED `RAPIDO`). Si al avanzar no queda en la celda esperada, abandona la ruta y sigue explorando desde ahí.

Corrida. El botón corto inicia una corrida nueva (el robot debe estar en el inicio, siempre en la misma celda y mirando al mismo lado); otro toque la detiene.

Página web. Puerto 7000 por WiFi. Durante una corrida **no llama al Bridge**: muestra la última lectura que tomó el robot y su última foto, para no quitarle tiempo ni la cámara. Fuera de corrida lee directamente los sensores. Incluye el panel de calibración de colores, la vista de la cámara y un dibujo en vivo de los ToF con lo que haría el robot.

8. Hardware

En simple: una batería de 7.4 V alimenta los motores y la placa. Todos los sensores comparten dos cables de datos (I²C). El chasis lo diseñamos nosotros en 3D, con medidas exactas, y lo imprimimos. Comprobamos por computadora que ninguna pieza choque con otra antes de imprimir.

Subsistema	Implementación
Alimentación	Batería 2S (6.0–8.4 V) → interruptor → L298N (V_S) y VIN del UNO Q (7–24 V, reguladores conmutados)
Lógica de potencia	Regulador 78M05 del L298N (necesita ≳ 7 V). Entradas compatibles con 3.3 V (V_IH ≥ 2.3 V)
Bus I²C	400 kHz, alimentado a 3.3 V para no exponer los pines del Q a *pull-ups* de 5 V. Lleva 3 × VL53L0X, TCS34725, una OLED SSD1306 y, opcional, un MPU-6050 (0x68)
Pantalla	OLED 128 × 64 con driver propio. Muestra estado, conteos y un contador de *heartbeat* de la comunicación entre los procesadores
Chasis	OpenSCAD paramétrico de dos pisos, FDM (Bambu X2D), insertos de latón M3 y pedestal impreso con inserto M4. Interferencias verificadas por intersección CSG
Cambio de placa	Mismo mapa de pines en Q y R3 vía Proto Shield; solo cambia el puente de VCC (3V3 ↔ 5V)

Voltaje en los motores: el L298N es bipolar (Darlington) y cae ≈ 2–3 V. Con 7.4 V llegan ≈ 4.5–5 V a los motores. Es la principal ineficiencia del diseño; un puente MOSFET (TB6612FNG, DRV8833) la eliminaría.

9. Parámetros calibrables

Valores tal como están en `sketch.ino` y `main.py`.

Sketch (STM32)

Parámetro	Valor	Qué controla
`CELDA`	180 mm	Avance de una celda
`RANGO`	1000 mm	Con el frontal por debajo avanza por distancia; más lejos, por tiempo
`MS_CELDA`	600 ms	Tiempo por celda cuando no hay pared a la vista
`MS_90`	250 ms	Giro de 90° por tiempo (solo de respaldo, si el MPU no responde)
`VEL_EXPLORA` / `VEL_LENTO` / `VEL_GIRO` / `VEL_MIN`	140 / 100 / 130 / 80	PWM de 0 a 255
`PARED` / `PARED_F`	120 / 55 mm	Pared lateral / pared al frente (frente en mm reales: requiere el frontal calibrado)
`D_FRENTE` / `D_CERCA`	35 / 5 mm	Alto al frente / retrocede si baja de 30 mm (mm reales, frontal calibrado)
`FRENA_K`	2 PWM/mm	Frenado proporcional al acercarse
`SALTO` / `TRAS_SALTO`	120 / 124 mm	Cambio brusco / avance posterior para centrarse en la celda abierta
`D_LADO_MIN`	90 mm	Solo corrige el centrado si un lateral lee menos
`D_LADO`	50 mm	Distancia lateral con el robot centrado
`KC` / `KD`	0.8 / 0.3	Centrado (posición / velocidad de acercamiento)
`KG`	2.0 PWM/°	Sostener rumbo en recta (con MPU-6050)
`CHOQUE_G` / `LEVANTA_G`	0.6 g / 0.8 g	Golpe horizontal que cuenta como choque / vertical debajo de la cual está levantado (150 ms)
`RETRO_MS` / `ATORADO_MS`	250 / 400 ms	Retroceso tras un choque / giro sin moverse 2° que cuenta como atorado

Parámetro	Valor	Qué controla
`ALIN_MIN`	3°	Con MPU, desvío desde el que endereza en su lugar antes de avanzar
`ADELANTO`	6°	Corta el giro antes por la inercia (si se pasa, subir)
`AJUSTE_DER`	−10	Diferencia de PWM en recta; negativo = más fuerza al motor izquierdo (en pista se iba a la izquierda)
`INV_IZQ` / `INV_DER`	true / true	Invierte el sentido del motor por software
`BRILLO_MIN`, `DIST_MAX`	1.3 / 0.02	Filtros del sensor de color

Python (Linux)

Parámetro	Valor	Qué controla
`FRENTE_LIBRE`	130 mm	El frente cuenta como salida (mm reales: a la mitad entre ~35-45 y ~215-225)
`VOTO_MAX`	5	Tope de votos por pared
`Y_COLOR`	36 mm	Del eje al sensor de color
`UMBRAL_MIN` / `UMBRAL_MAX`	100 / 220	Límites del umbral de gris de Otsu
`ZONA_BORDES`	(0.45, 0.95)	Filas donde busca las bases de las paredes
`ANG_MIN` / `ANG_MAX`	4° / 20°	Corrige el rumbo desde / descarta arriba de
`camcal` (web)	expo 0, x_0 160, k 0.008	Exposición, columna “derecho”, radianes por píxel; experimental: fila → mm. En `/app/datos/camara.json`
`EXPERIMENTAL`	False	`True` activa la distancia y las aberturas por cámara (técnicas 4 y 5)
`COLS` / `SIG`	5 columnas / 60-240 mm	Experimental: columnas de distancia / tramo de la celda siguiente
`ZONA_PASO`	(0.30, 0.55, 0.70, 1.00)	Piso por donde va a pasar, en fracción de la imagen
`BLANCO_MAX`	0.15	Fracción de blanco que activa el avance lento
Colores de referencia	de fábrica en `REFS`	Se miden en el panel web; se guardan en `/app/datos/colores.json`

10. Glosario

Término	Significado
MCU	Microcontrolador: computadora pequeña y puntual para controlar hardware
MPU	Microprocesador: computadora grande que corre un sistema operativo (Linux)
RPC	Llamada a procedimiento remoto: un procesador le pide a otro que ejecute una función
I²C	Bus de 2 cables (SDA datos, SCL reloj) que comparten varios sensores
ToF	*Time of Flight*: mide distancia por el tiempo que tarda la luz en ir y volver
PWM	Prender y apagar muy rápido para regular la potencia; 0 = apagado, 255 = máximo
Puente H	Circuito que deja girar un motor en ambos sentidos
Lazo abierto / cerrado	Sin / con medición del resultado para corregir
Control P	Corrige en proporción al error
Encoder	Sensor que cuenta las vueltas de la rueda
IMU / MPU-6050	Sensor de giro y aceleración; aquí se usa su giroscopio para medir el ángulo de los giros
BFS	Búsqueda en anchura: encuentra el camino más corto entre dos celdas
Rumbo	Hacia dónde apunta el robot, en grados
HSV	Forma de describir un color por tono, saturación y brillo
Obturador global	Todos los píxeles se exponen al mismo tiempo: la imagen no se tuerce con el movimiento
Otsu	Método que elige solo el umbral que mejor separa dos grupos de gris (aquí, piso y pared)
Punto de fuga	Donde se juntan en la imagen dos líneas que en la realidad son paralelas (las paredes del pasillo)
Bridge	Servicio que pasa las llamadas (RPC) entre Linux y el STM32

Término	Significado
CSG	Operaciones booleanas entre sólidos 3D (unión, resta, intersección)

11. Evolución del diseño

La bitácora completa de cambios, problemas y soluciones está en `historial_cambios.md`. Los cambios más importantes desde la V1:

- **V1:** movimiento por tiempo, sin encoders, con *flood fill* sobre un mapa de 16 × 16 celdas y respaldo en UNO R3 (ATmega328P).
- **Sensores:** 3 × VL53L0X reasignados de dirección I²C y sensor de color TCS34725 con calibración desde el panel web.
- **Cámara:** primero detectaba paredes para anticipar caminos; después buscó también el amarillo; con la cámara monocromática OV9281 detecta blanco (pared) con umbral automático y corrige el rumbo con el punto de fuga del pasillo; la versión experimental además mide distancias y aberturas.
- **Avance por distancia:** el frontal mide la celda, para a 35 mm reales y retrocede si queda muy cerca; sin pared va por tiempo.
- **Cambios bruscos** de los ToF (`SALTO`): el robot se centra en la celda abierta y la escanea.
- **Decisión:** pasó de *flood fill* con paredes de la cámara, y luego de reglas de pared, a una máquina de estados `SENSA → DECIDE → GIRA → AVANZA` con prioridad frente, izquierda, derecha; después, regla de la mano izquierda pura (izquierda, frente, derecha).
- **Mapa:** votos ±5 por pared y colores por celda; ruta corta por BFS en la siguiente corrida.
- **Meta:** el STM32 para solo al confirmar el amarillo, sin esperar a Python.
- **Giros con MPU-6050** opcional: rumbo absoluto y rumbo sostenido en recta; sin él, por tiempo.
- **Web:** durante la corrida no usa el Bridge para no quitarle tiempo al robot.

12. Control de versiones

Revisión	Fecha	Qué cambió en el código
A	19-oct a 9-nov	Iteración A: UNO R3 / Nano con firmware *bare-metal* (`atmega328p/`, respaldo)
B1	9 a 11-nov	UNO Q: sketch + Python, mapa por votos, avance por distancia, máquina de estados
B2	11-nov	MPU-6050 opcional; se agregan 2 sketches: `sketch/` (con MPU) y `sketch_sin_mpu/` (sin MPU)
B3	11-nov	Cámara OV9281 monocromática: Otsu, rumbo por punto de fuga; `python_experimental/`
B4	11-nov	Exploración completa (meta al centro), calibración de ToF a 100 mm, control desde la web, cronómetro
B5	11-nov	MPU completo: alinear, giros verificados, choques, giro atorado, robot levantado
B6	11-nov	Se retira `sketch_sin_mpu/` del repositorio: el robot lleva MPU-6050. Queda un solo sketch, `sketch/`
B7	**11-nov**	**Se retira `python_experimental/` (era el mismo `main.py` con `EXPERIMENTAL = True`). Queda un solo Python**

Recuperar la versión sin MPU. La última versión de `uno_q/sketch_sin_mpu/` está en el commit `2e01996` del repositorio. Se recupera con:

```
git show 2e01996:uno_q/sketch_sin_mpu/sketch.ino > sketch_sin_mpu.ino
```

(o en GitHub, abriendo el commit `2e01996` y la carpeta `uno_q/sketch_sin_mpu/`; ahí también está `uno_q/python_experimental/`). Funciona con el mismo Python y la misma página web; gira y avanza por tiempo (`MS_90`, `MS_CELDA`). Además, `uno_q/sketch/` sigue funcionando sin MPU si no

lo detecta al arrancar.